

FLOWDAR

Specifications Backend — v3

Document de reference pour Mr Ebanga Arnaud — Angular Talent Lab 2026

Ce document est a destination de l'encadreur (Mr Ebanga Arnaud) qui developpe le backend. Le frontend Angular (Nlend Max) consomme uniquement l'API REST exposee — il ne touche pas au backend.

1. Stack technique recommandee

Composant	Technologie	Justification
Runtime	Node.js 20+ LTS	Performances async pour cron jobs et requetes GPS en parallele (Promise.all)
Framework API	Express.js	Simple, bien documente, parfait pour les endpoints REST
Base de donnees	PostgreSQL 16+ + PostGIS	Requetes geospaciales (altitude, proximite eau, polygones)
Scraping	Puppeteer + Cheerio	Puppeteer pour ONACC (site JS dynamique), Cheerio pour HTML statique
IA (Could)	Gemini API	Analyse signalements texte + resumes (si temps disponible)
Taches planifiees	node-cron	Jobs automatiques toutes les 15-30 min
Requetes paralleles	Promise.all	Requetes meteo GPS simultanees sans bloquer
Deploiement	Railway	Gratuit pour projets de formation, PostgreSQL integre

2. Modele de donnees PostgreSQL

Table : zones_a_risque

Champ	Type	Description
id	UUID PK	Identifiant unique
nom_quartier	VARCHAR(100)	Ndokotti, Bepanda, Makepe, Logpom, PK8-PK14, Ndog-Bong, Melen
lat	DECIMAL(10,7)	Latitude GPS centroide
lng	DECIMAL(10,7)	Longitude GPS centroide
polygone	GEOMETRY(POLYGON)	Contour PostGIS

Champ	Type	Description
altitude_m	INTEGER	Altitude metres (Google Elevation API)
proximite_eau_m	INTEGER	Distance metres cours d'eau (OpenStreetMap)
niveau_risque_base	ENUM	faible / moyen / eleve
seuil_score	INTEGER	Seuil declenchement (0-100) propre a ce quartier
nb_occurrences	INTEGER DEFAULT 0	Nombre d'inondations enregistrees
source	ENUM	onacc / historique_terrain / signalement_citoyen

Table : alertes

Champ	Type	Description
id	UUID PK	Identifiant unique
type	ENUM	preventive / active
source	ENUM	auto / citoyen / onacc — NB : pas de twitter/facebook (Won't)
zone_id	UUID FK	Lien vers zones_a_risque
score	INTEGER 0-100	Score calcule et plafonne (voir section 3)
niveau	ENUM	leger / moyen / dangereux
heure_debut	TIMESTAMP	Creation de l'alerte
heure_prevue	TIMESTAMP NULL	Heure prevue (preventives uniquement)
nb_confirmations	INTEGER DEFAULT 0	Confirmations citoyennes
statut	ENUM	actif / resolu / en_resolution
photo_url	VARCHAR NULL	URL Firebase Storage
firestore_id	VARCHAR	ID Firestore pour sync temps reel

Combinaisons valides type x niveau : PREVENTIVE → léger ou moyen uniquement. ACTIVE → léger, moyen ou dangereux. Un score previsionnel > 85 cree une alerte preventive en 'moyen' ; elle passera en 'dangereux' uniquement quand elle devient active.

Table : historique_meteo_alertes

Champ	Type	Description
id	UUID PK	Identifiant unique
zone_id	UUID FK	Quartier
date	TIMESTAMP	Date et heure
pluie_mm_h	DECIMAL	Precipitations mm/h (requete GPS individuelle)

Champ	Type	Description
humidite_pct	INTEGER	Humidite sol %
pluie_cumul_3h	DECIMAL	Pluie cumulee 3h
score_calcule	INTEGER	Score calcule (0-100, plafonne)
alerte_generee	BOOLEAN	Alerte creee ?

Table : signalements_bruts

Champ	Type	Description
id	UUID PK	Identifiant unique
texte_brut	TEXT	Contenu brut saisi par le citoyen ou scrape (ONACC uniquement)
lat_signal	DECIMAL NULL	GPS du signalement si zone inconnue
lng_signal	DECIMAL NULL	GPS du signalement si zone inconnue
source	ENUM	citoyen / onacc — NB : pas de twitter/facebook (Won't)
analyse_gemini	JSONB NULL	Resultat analyse Gemini (si disponible)
alerte_generee_id	UUID NULL FK	Alerte generee si valide
created_at	TIMESTAMP	Date collecte

Table : confirmations

Champ	Type	Description
id	UUID PK	
alerte_id	UUID FK	Alerte confirmee
user_uid	VARCHAR	UID Firebase Auth
type	ENUM	confirme / resolu
created_at	TIMESTAMP	Heure

3. Logique de calcul du score — CORRIGE v3

*Correction v3 (point 2 Claude Code) : le facteur meteo est plafonne a $\min(\text{pluie}/\text{seuil}, 1) * 40$ pour garantir un score total toujours entre 0 et 100.*

Requetes meteo GPS par quartier

```
// Une requete distincte par zone, toutes en parallele :
```

```
const resultats = await Promise.all(
  zones.map(zone => fetch(
    `/weather?lat=${zone.lat}&lon=${zone.lng}&appid=${OPENWEATHER_KEY}`
  ))
);
```

Formule du score (0-100, plafonne)

Facteur	Poids	Formule — v3 corrigee
Meteo temps reel	40%	$\min(\text{pluie_mm_h} / \text{zone.seuil_mm_h}, 1) * 40 \leftarrow \text{PLAFONNE}$
Historique	30%	$(\text{nb_alertes_conditions_similaires} / \text{nb_total_mesures}) * 30$
Signalements citoyens	20%	$\min(\text{nb_signalements_actifs_1h} / 3, 1) * 20$
Geographie	10%	$\min(\text{score_geo}, 1) * 10$ (score_geo = f(altitude, proximite_eau))

```
// Score final — toujours entre 0 et 100 :
const score = Math.round(
  min(pluie_mm_h / zone.seuil_mm_h, 1) * 40 +
  ratio_historique * 30 +
  min(nb_signalements / 3, 1) * 20 +
  min(score_geo, 1) * 10
);
// score est naturellement plafonne a 100 (somme des max de chaque facteur)
```

Synchronisation PostgreSQL vers Firestore — CLARIFIE v3

La synchronisation se fait via un appel explicite a Firebase Admin SDK a la fin de chaque cron job score-update et zone-discovery. Ce n'est pas un trigger PostgreSQL automatique — c'est un appel programmatique delibere apres chaque ecriture en base.

```
// A la fin de score-update et zone-discovery :
await admin.firestore()
  .collection('alertes')
  .doc(alerte.firestore_id)
  .set({
    id: alerte.id,
    zone_id: alerte.zone_id,
    score: alerte.score,
    niveau: alerte.niveau,
    statut: alerte.statut,
    lat: zone.lat,
    lng: zone.lng
  });
// Angular reçoit la mise a jour via onSnapshot automatiquement
```

4. Endpoints API

Alertes

Methode	Endpoint	Description	Reponse
GET	/api/alertes	Alertes actives	Array avec score, niveau, lat, lng, type
GET	/api/alertes/:id	Detail alerte	Alerte + decomposition score 4 facteurs + confirmations
POST	/api/alertes/signaler	Signalement citoyen	Zone connue (zone_id) OU inconnue (lat/lng GPS)
POST	/api/alertes/:id/confirmer	Confirmer alerte	nb_confirmations +1
POST	/api/alertes/:id/resoudre	Marquer resolue	Statut -> resolu
GET	/api/alertes/historique/:quartier	Historique	Array alertes passees + score moyen

Zones, scores, meteo, itineraire

Methode	Endpoint	Description	Reponse
GET	/api/zones-a-risque	Toutes les zones	Array avec lat, lng, seuil_score, niveau_risque_base
GET	/api/scores	Scores actuels toutes zones	Array { zone_id, score, niveau }
GET	/api/meteo/actuelle	Meteo actuelle par zone	Array { zone_id, pluie_mm_h, humidite }
GET	/api/meteo/previsions	Previsions 6h par zone	Array { zone_id, heure_prevue, score_previsionnel }
POST	/api/itineraire	Zones actives a eviter	Array coordonnees GPS zones alertees (passe a Google Maps cote Angular)

Reponse enrichie GET /api/alertes/:id — decomposition score

L'endpoint detail doit retourner la decomposition du score en 4 facteurs pour que l'ecran Angular puisse l'afficher a l'utilisateur.

```
// Reponse attendue par Angular pour GET /api/alertes/:id :
```

```
{
  id, type, source, zone_id, score, niveau, statut,
  heure_debut, nb_confirmations,
  score_detail: {
    meteo: 32,          // contribution facteur meteo (sur 40)
    historique: 18,     // contribution facteur historique (sur 30)
    citoyens: 14,       // contribution facteur signalements (sur 20)
    geographie: 7       // contribution facteur geo (sur 10)
  }
}
```

5. Cron Jobs

Job	Frequence	Ce qu'il fait
score-update	Toutes les 30 min	Requetes meteo GPS par zone (Promise.all), calcul score plafonne, creation/MAJ alertes, ecriture PostgreSQL, sync Firestore explicite
zone-discovery	Toutes les 30 min	Verifie signalements hors zones connues. Si 5+ sur meme GPS en < 1h : cree nouvelle zone, sync Firestore
seuil-update	Toutes les 24h	Analyse historique_meteo_alertes, ajuste seuil_score par zone selon occurrences reelles
scraper-onacc	Toutes les heures	Puppeteer scrape onacc.cm, extrait bulletins officiels (pas de Twitter/Facebook — Won't)
auto-resolve	Toutes les heures	Alertes sans confirmation > 3h : statut -> resolu, sync Firestore

6. Variables d'environnement

Variable	Obtention
OPENWEATHER_API_KEY	openweathermap.org — compte gratuit
GOOGLE_ELEVATION_API_KEY	console.cloud.google.com
GEMINI_API_KEY	aistudio.google.com (si fonctionnalite Could implementee)
DATABASE_URL	Railway lors du deploiement
FIREBASE_SERVICE_ACCOUNT	Console Firebase -> Parametres -> Comptes de service
FRONTEND_URL	URL Firebase Hosting Angular (pour CORS)

7. Checklist de livraison backend

1. API REST operationnelle (tous endpoints section 4)

2. Score plafonne : $\min(\text{pluie/seuil}, 1) * 40$ implemente
3. Requetes meteo GPS individuelles par quartier (Promise.all)
4. Decomposition score en 4 facteurs retournee dans GET /api/alertes/:id
5. Sync Firestore explicite a la fin de score-update et zone-discovery
6. Source enum signalements_bruts : citoyen et onacc uniquement (pas twitter/facebook)
7. Combinaisons type x niveau respectees (preventive ne peut pas etre dangereux)
8. Zones precharges : Ndokotti, Bepanda, Makepe, Logpom, PK8-PK14, Ndog-Bong, Melen
9. URL API publique communiquee a Nlend Max pour integration Angular